

AWS EC2 Based Student Registration Portal

Executive Summary

This project demonstrates the deployment of a Student Registration Portal on AWS Cloud using Amazon EC2, Apache, PHP and MariaDB. The application allows students to submit registration information through a web interface and stores the data in a relational database.

Problem Statement

Many organizations require a centralized system to collect and manage student information. This project provides a cloud-hosted registration portal capable of collecting and storing student data securely.

Project Objectives

- Deploy a web application on AWS EC2
- Configure Apache and PHP
- Create and manage MariaDB database
- Enable frontend-backend integration
- Demonstrate cloud deployment and database connectivity

Technology Stack

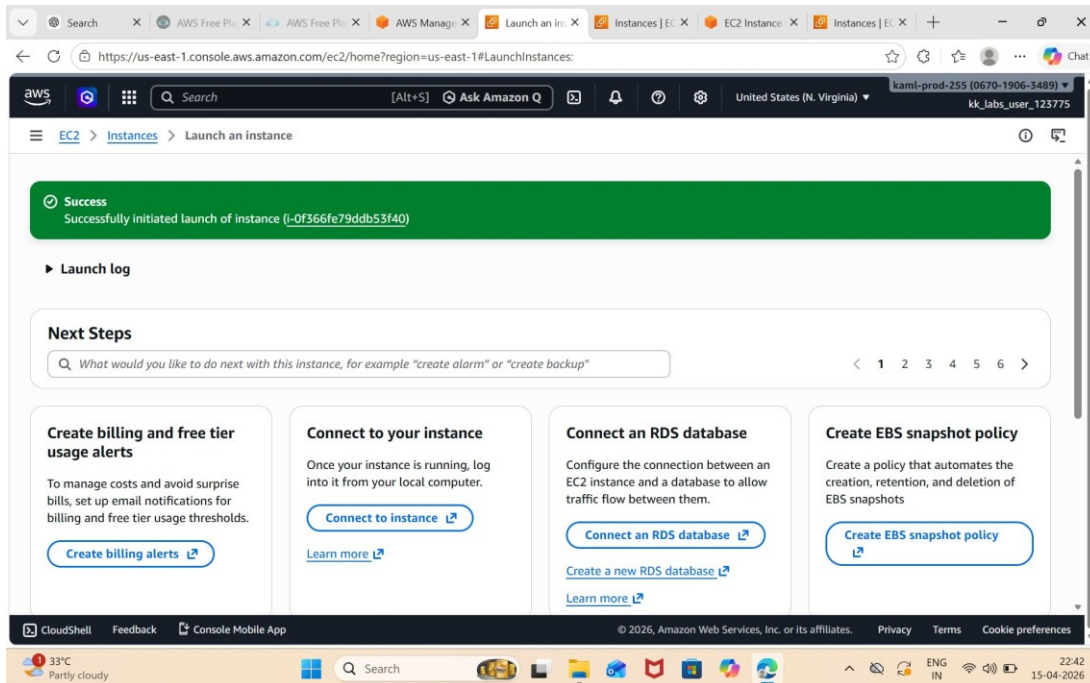
AWS EC2, Amazon Linux, Apache HTTP Server, PHP, MariaDB, HTML, CSS, SQL, Linux

System Architecture

The project follows a 2-Tier Architecture. Apache and PHP run on AWS EC2 as the application layer. MariaDB acts as the database layer storing student records.

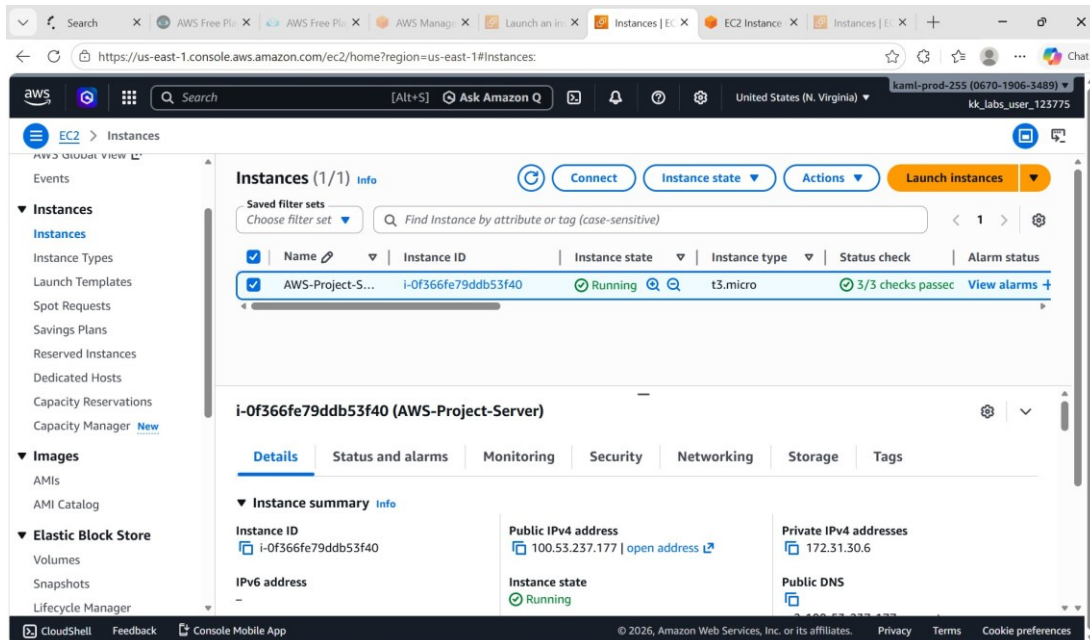
Step 1 - EC2 Launch Success

Screenshot evidence for step 1 - ec2 launch success.



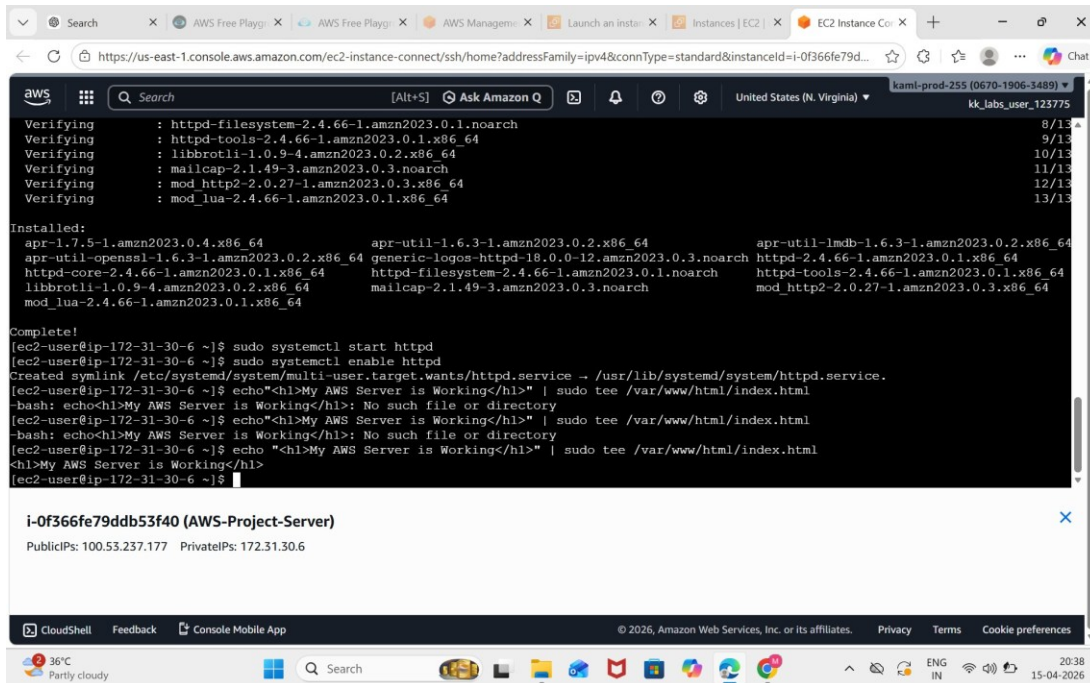
Step 2 - EC2 Instance Running

Screenshot evidence for step 2 - ec2 instance running.



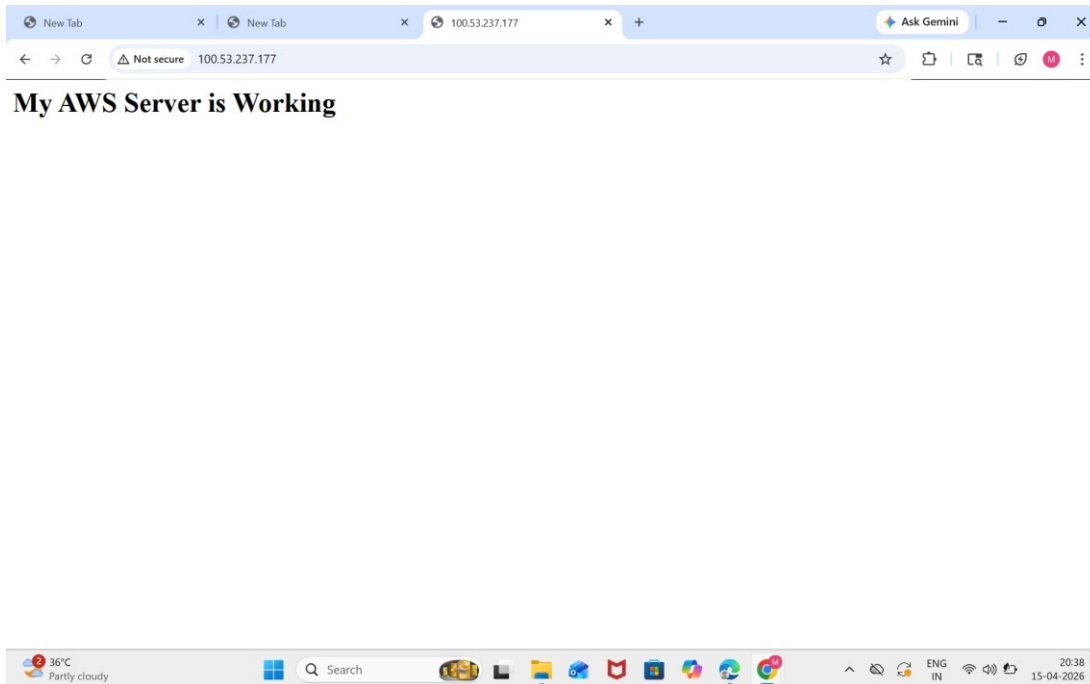
Step 3 - Apache Server Setup

Screenshot evidence for step 3 - apache server setup.



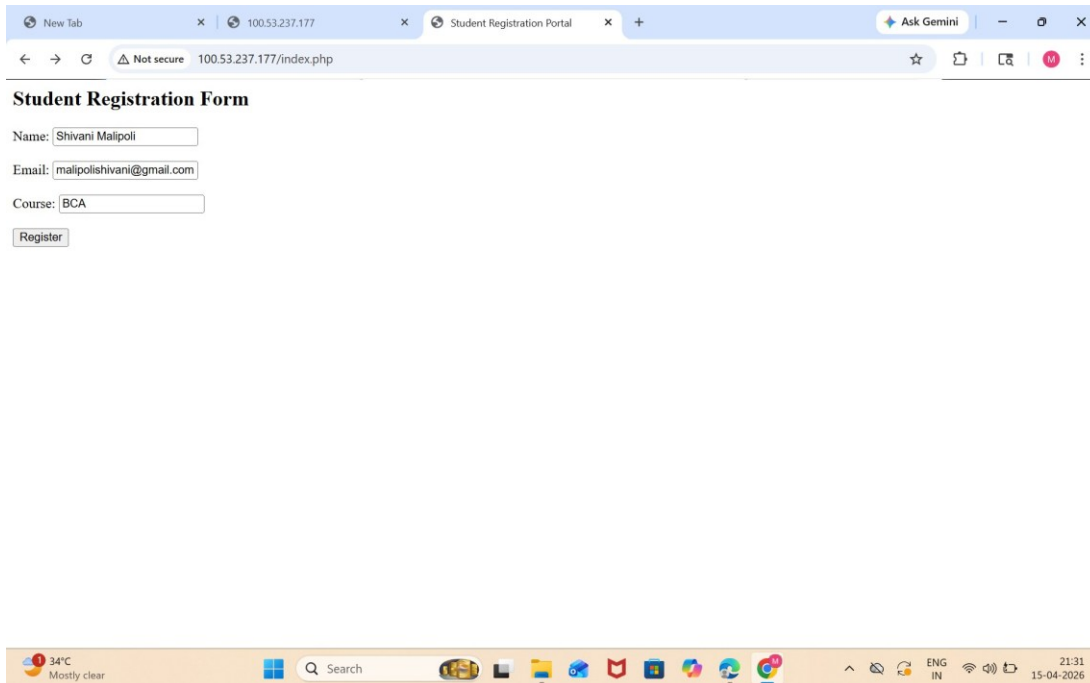
Step 4 - Apache Verification

Screenshot evidence for step 4 - apache verification.



Step 5 - Application Deployment

Screenshot evidence for step 5 - application deployment.



Step 6 - MariaDB Setup

Screenshot evidence for step 6 - MariaDB setup.

```
[ec2-user@ip-10-0-0-134 ~]$ sudo mysql -u root -p
Enter password:
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 15
Server version: 10.5.22-MariaDB MariaDB Server

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]> show databases;
+-----+
| Database |
+-----+
| information_schema |
| mysql |
| performance_schema |
| studentdb |
| sys |
+-----+
5 rows in set (0.002 sec)

MariaDB [(none)]> use studentdb;
Database changed
MariaDB [studentdb]> show tables;
+-----+
| Tables_in_studentdb |
+-----+
| students |
+-----+
1 row in set (0.001 sec)

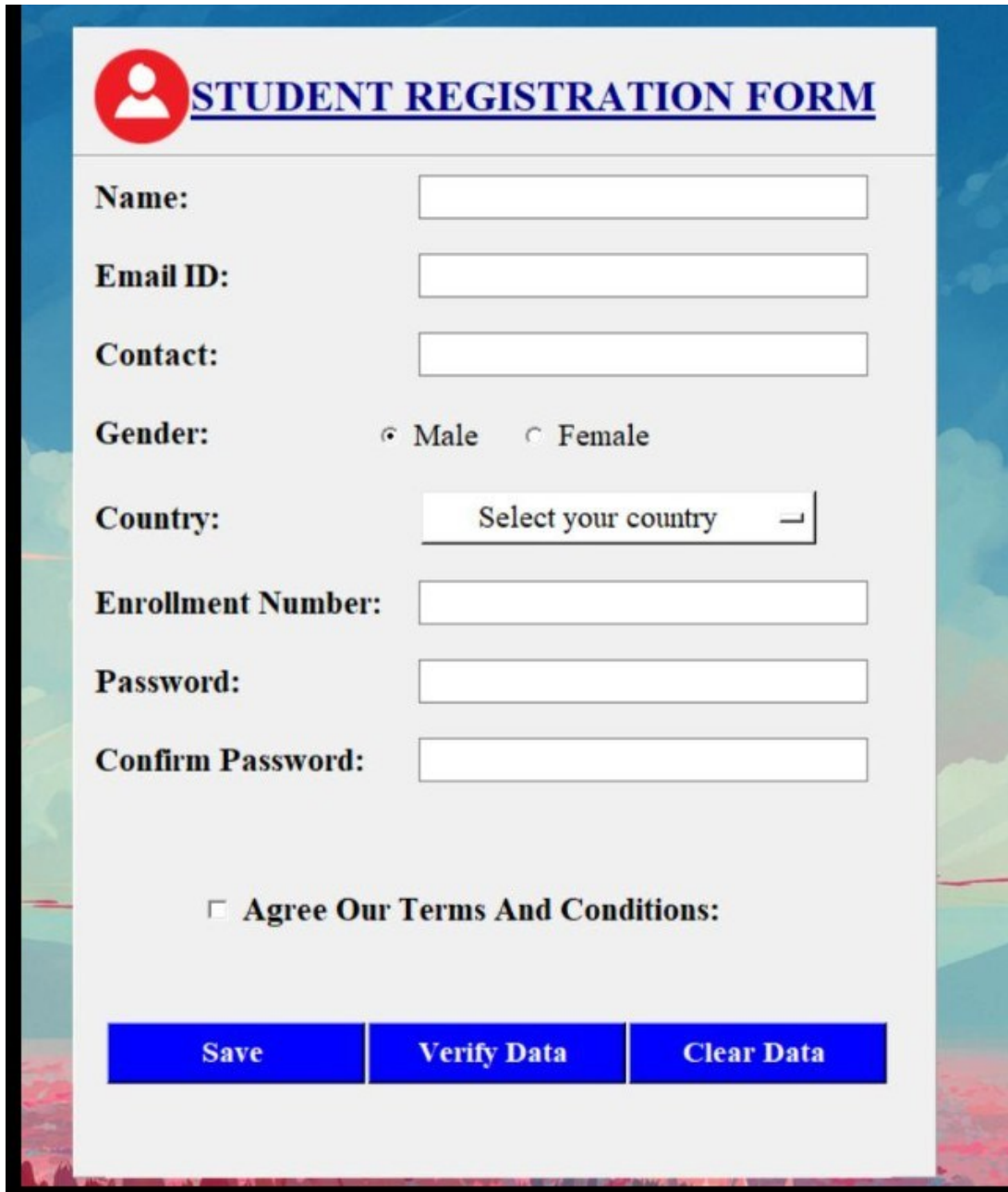
MariaDB [studentdb]> describe students;
+-----+
| Field | Type | Null | Key | Default | Extra |
+-----+
| id | int(11) | NO | PRI | NULL | auto_increment |
| name | varchar(100) | YES | | NULL | |
| email | varchar(100) | YES | | NULL | |
| contact | varchar(15) | YES | | NULL | |
| course | varchar(50) | YES | | NULL | |
| created_at | timestamp | YES | | current_timestamp() | |
+-----+
6 rows in set (0.001 sec)

MariaDB [studentdb]> select * from students;
+-----+
| id | name | email | contact | course | created_at |
+-----+
| 1 | Shivani | shivani.mali@example.com | 9876543210 | BCA | 2025-05-15 12:45:10 |
| 2 | Rahul | rahul.kumar@example.com | 9123456780 | BSc IT | 2025-05-15 12:48:33 |
+-----+
2 rows in set (0.001 sec)

MariaDB [studentdb]>
```

Step 7 - Registration Form UI

Screenshot evidence for step 7 - registration form ui.



The screenshot displays a web form titled "STUDENT REGISTRATION FORM" with a red circular logo containing a white person icon. The form is set against a light gray background with a blue border. It includes several input fields: "Name:", "Email ID:", "Contact:", "Enrollment Number:", "Password:", and "Confirm Password:". The "Gender:" field has radio buttons for "Male" and "Female", with "Male" selected. The "Country:" field is a dropdown menu showing "Select your country". Below the input fields is a checkbox labeled "Agree Our Terms And Conditions:". At the bottom, there are three blue buttons: "Save", "Verify Data", and "Clear Data".

STUDENT REGISTRATION FORM

Name:

Email ID:

Contact:

Gender: ☒ Male ☐ Female

Country:

Enrollment Number:

Password:

Confirm Password:

☐ **Agree Our Terms And Conditions:**

Save **Verify Data** **Clear Data**

Step 8 - Database Connectivity Testing

Screenshot evidence for step 8 - database connectivity testing.

The screenshot shows a terminal window within an AWS EC2 instance. The terminal output includes the following commands and results:

```
Query OK, 0 rows affected (0.008 sec)

MariaDB [studentdb]> exit;
Bye
[ec2-user@ip-172-31-30-6 ~]$ sudo nano /var/www/html/submit.php
[ec2-user@ip-172-31-30-6 ~]$ sudo mysql
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 7
Server version: 10.5.29-MariaDB MariaDB Server

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]> SELECT * FROM students;
ERROR 1046 (3D000): No database selected
MariaDB [(none)]> USE studentdb;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
MariaDB [studentdb]> SELECT * FROM students;
Empty set (0.001 sec)

MariaDB [studentdb]>
```

Below the terminal window, a metadata box for the EC2 instance is visible, showing the instance ID `i-Of366fe79ddb53f40` (AWS-Project-Server) and its Public IP `100.53.237.177`.

Database Verification and Backend Testing

This screenshot demonstrates the successful verification of the MariaDB database connectivity on the AWS EC2 instance. After configuring the database and deploying the Student Registration Portal, SQL commands were executed to validate the connection between the application layer and the database layer.

The student db database was successfully selected, and SQL queries were executed to verify that the database was accessible from the server environment. The successful execution of the SELECT query confirmed that the MariaDB service was running correctly and that the application could communicate with the database without connectivity issues.

This testing phase played a crucial role in validating the backend functionality of the project. It ensured that the database configuration, user permissions, and server settings were properly established. By performing these validation checks, the reliability and stability of the Student Registration Portal were confirmed before production deployment.

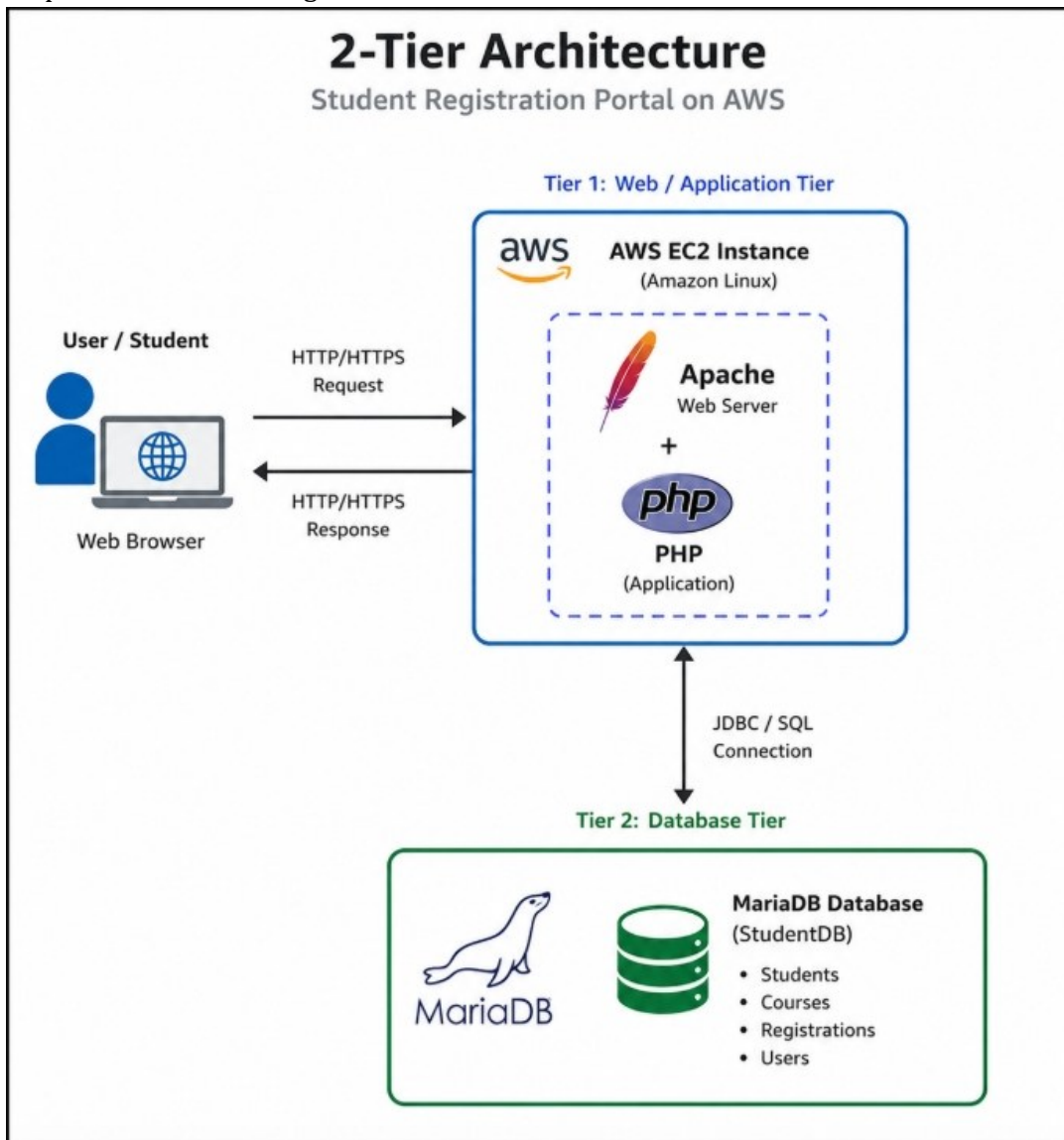
Key Outcomes:

- Verified MariaDB database accessibility.
- Confirmed successful database selection and query execution.

- Validated communication between the application and database layers.
- Ensured proper backend configuration on AWS EC2.
- Demonstrated successful database integration within the 2-Tier Architecture.

This verification process confirms that the Student Registration Portal can reliably store, retrieve, and manage student information within the cloud-hosted environment.

Step9 - Architecture Diagram



Database Design

Table: students

Fields: id, name, email, contact, course, created_at

Primary Key: id

Application Workflow

User opens the registration form, enters details, submits the form, PHP processes the request, and data is stored in MariaDB.

AWS Services Used

Amazon EC2 for hosting, Security Groups for access control, Public IP for web access, Linux environment for deployment.

Implementation Steps

1. Launch EC2
2. Configure Security Group
3. Install Apache
4. Install PHP
5. Install MariaDB
6. Create Database
7. Deploy Application
8. Test Connectivity
9. Verify Data Storage

Testing Results

Application successfully served through EC2 public IP. Student records were inserted and verified in MariaDB.

Challenges and Solutions

Resolved package installation issues, database connection configuration, Apache service configuration and deployment verification.

Key Learning Outcomes

AWS EC2 administration, Linux commands, Apache configuration, PHP integration, MariaDB management, SQL operations and cloud deployment.

Future Enhancements

User authentication, HTTPS, AWS RDS migration, responsive UI, email notifications, load balancing.

GitHub Repository Structure

README.md
screenshots/
source-code/
database/
docs/

Interview Talking Points

Explain EC2 deployment, Apache setup, PHP-MariaDB integration, architecture design, database testing and cloud hosting workflow.

Conclusion

This project demonstrates practical cloud deployment skills and full-stack integration using AWS, making it a strong portfolio project for fresher software, cloud and web development roles.